



Universidad Simón Bolívar

Decanato de Estudios Profesionales

Departamento de Computación y Tecnología de la Información

CI2692 Laboratorio de Algoritmos y Estructuras de Datos II

Trimestre Abril-Julio 2026

Profesores: Leonardo López - Kevin Oporto

Junio 2026

Proyecto I: Motor de Visibilidad y Ordenamiento Estructural en 2D

1. Introducción

En los motores gráficos bidimensionales y sistemas de simulación por rejillas, la pantalla se compone de múltiples elementos espaciales (entidades, estructuras, personajes) distribuidos en un plano coordenado discreto. Para renderizar la escena de forma eficiente sin colapsar el procesador, el sistema debe realizar dos tareas críticas sobre arreglos de datos: el filtrado de estructuras fuera del campo visual (*Culling*) y el ordenamiento de atrás hacia adelante (*Algoritmo del Pintor*) para asegurar que los objetos cercanos se dibujen correctamente sobre los lejanos.

El objetivo de este proyecto es diseñar e implementar un simulador de gestión de escena que optimice la tubería de renderizado utilizando tres estrategias de ordenamiento aplicadas sobre arreglos de objetos: `insertionSort`, `mergeSort` y `quickSort`.

2. Preliminares y Reglas del Motor

2.1. Propiedades y Dimensiones del Tablero y la Cámara

- **El Tablero (Mapa Global):** Representa las fronteras del mundo y define el límite máximo permitido para posicionar elementos. El mundo consta de un plano discreto bidimensional con dimensiones fijas de **100 x 100 posiciones enteras**. Cualquier punto con coordenadas fuera de los rangos $X \in [0, 99]$ e $Y \in [0, 99]$ se considera inválido.
- **La Cámara:** Tiene dimensiones fijas de **16 unidades de ancho por 9 de alto**. Cuando la cámara se posiciona en un centro coordenado entero (X_{cam}, Y_{cam}) , los límites fronterizos inclusivos para el escaneo visual se calculan automáticamente de la siguiente forma:

$$xMin = X_{cam} - 8$$

$$xMax = X_{cam} + 8$$

$$yMin = Y_{cam} - 4$$

$$yMax = Y_{cam} + 4$$

Nota: Si la cámara se posiciona cerca de los bordes del mundo, el programa debe truncar estos límites para que no excedan los extremos del tablero (0 o 99).

2.2. Cuadrantes (Regiones)

Para evitar un escaneo lineal ineficiente de todo el mapa cada vez que la cámara cambia de lugar, el tablero de 100×100 se divide estáticamente en una cuadrícula de **10 x 10 regiones (100 cuadrantes en total)**. Cada región mide exactamente 10×10 unidades de la rejilla.

Al situar la cámara, el sistema identifica en qué cuadrantes caen sus límites físicos. El motor realizará la búsqueda analítica **únicamente dentro del cuadrante actual donde reside el centro de la cámara y sus cuadrantes aledaños directamente afectados (un bloque máximo de hasta 3 x 3 regiones)**. Las regiones restantes del mundo quedan completamente omitidas de la búsqueda, optimizando el rendimiento global del sistema.

2.3. Pseudocódigo del Filtro de Visibilidad

A continuación se presenta el algoritmo base que debe utilizar el motor para determinar si un elemento espacial se encuentra dentro del rango de visión calculado para la cámara:

```
procedimiento verificarVisibilidad(in e: SpatialElement, in cam: Camara,
out vis: Boolean)
{Pre: e inicializado Y cam inicializada con sus límites xMin, xMax, yMin,
yMax}
inicio
    val dentroX = (e.x ≥ cam.xMin) Y (e.x ≤ cam.xMax)
    val dentroY = (e.y ≥ cam.yMin) Y (e.y ≤ cam.yMax)

    si (dentroX Y dentroY) entonces
        vis := True
    sino
        vis := False
    fin si
fin
{Post: vis = True ⇔ (e está dentro de los límites coordenados inclusivos
de cam)}
```

2.4. Criterio Estricto de Ordenamiento de Escena

Una vez que el motor determina cuáles elementos están dentro del marco de la cámara, estos deben ser consolidados en un arreglo plano temporal y ordenarse de menor a mayor bajo los siguientes criterios jerárquicos:

1. **Criterio Principal (Coordenada Z):** Se ordenan de menor a mayor según su valor numérico **z** (tipo **Double**, manejando un solo decimal, con valores válidos entre 0,0 y 10,9). Los objetos con menor **z** se pintan primero (fondo).

2. **Criterio de Desempate (Coordenada Y):** Si dos o más elementos poseen exactamente el mismo valor de z , se evalúa su coordenada vertical ' y '. Se ordenarán de mayor a menor respecto a ' y '. Esto simula la perspectiva del pintor: los objetos con mayor ' y ' están más arriba en la pantalla (más lejos) y se dibujan antes, mientras que los objetos con menor ' y ' están más abajo (más cerca) y se dibujan después, superponiéndose correctamente.

2.5. Criterios de Selección Automática del Algoritmo

El motor gráfico ejecutará de forma obligatoria y automática el algoritmo de ordenamiento adecuado basándose **únicamente en la cantidad de elementos visibles (N)** detectados tras el escaneo de la cámara en los cuadrantes involucrados:

- **insertionSort:** Se ejecuta automáticamente si el número de elementos visibles es menor o igual a 50 ($N \leq 50$).
- **mergeSort:** Se ejecuta automáticamente si el número de elementos visibles está entre 51 y 200 ($51 \leq N \leq 200$).
- **quickSort:** Se ejecuta automáticamente si el número de elementos visibles es mayor a 200 ($N > 200$).

3. Tipos de Datos (TADs) a Implementar

3.1. TAD Elemento Espacial (SpatialElement)

Representa una entidad individual colocada en la rejilla.

- **Campos:** `id` (Int), `x` (Int), `y` (Int), `z` (Double).

3.2. TAD Región Estructural (MapRegion)

Encapsula la gestión de un cuadrante específico de 10×10 del mapa utilizando arreglos nativos de tamaño fijo.

- **Campos:** `sectorId` (Int), `almacenamiento` (Array de SpatialElement), `cantidadElementos` (Int).
- **Invariante de Representación (Densidad Máxima):** Cada cuadrante tiene un factor de densidad (número máximo de elementos que puede contener) que **nunca debe superar el 0.8**. Si al intentar realizar una inserción en caliente se alcanza o supera este límite, el sistema **no modificará el arreglo**; en su lugar, abortará la operación mostrando un mensaje explícito en consola indicando que el elemento no puede ser ingresado debido a que se supera la densidad máxima permitida del sector.

3.3. TAD Gestor de Escena (SceneManager)

Orquestador principal del motor. Almacena las 100 regiones en un arreglo indexado contiguo. Contiene la lógica para calcular la intersección de la cámara con los cuadrantes

afectados, extraer los elementos válidos visibles en un arreglo temporal y disparar el ordenamiento de forma inmediata al situarse. Posee además un objeto histórico encargado de almacenar la lista de secuencias de IDs de cada ordenamiento válido.

3.4. Estructura del Historial de Movimientos en Memoria (Objeto Histórico)

Para dar soporte al registro cronológico de la simulación, el sistema mantendrá en memoria un objeto histórico basado en un **arreglo bidimensional**. El cual tendrá un total de 720 columnas (Cada posición de la cámara puede contener a lo sumo 9 cuadrantes con factor de densidad de 0.8) y n filas (número máximo de movimientos permitidos y se ingresa desde el cliente) este arreglo bidimensional estará lleno de '-1'.

Cada vez que la cámara se desplace a una posición donde existan objetos visibles ($N > 0$), el sub-arreglo temporal que contiene la secuencia de IDs ya ordenados por el motor se modificará la fila correspondiente al movimiento actual, reemplazando los '-1' por los IDs de los objetos visibles en el orden correcto. Si no hay objetos visibles, la fila se mantendrá completamente con '-1' y se omitirá del registro final.

4. Programa Cliente y Opciones del Menú (PruebaRender.kt)

Paso Base:

Antes de desplegar la interfaz interactiva, el sistema debe realizar la **Carga de la configuración del mapa**. El programa leerá el archivo de texto base (objetos.txt) estructurado como líneas de tipo `id,x,y,z` y poblará los arreglos estáticos de las 100 regiones correspondientes según su ubicación física en la cuadrícula. Si este paso falla o el archivo no existe, la aplicación terminará con un mensaje de error.

Una vez cargada la infraestructura en memoria, se ejecutará el programa `PruebaRender.kt`. Para ejecutarlo, debe crear un archivo Bash ejecutable llamado `runPruebaRender.sh n`, donde n es el número máximo de movimientos permitidos. La línea de comando que lo ejecuta es de la siguiente manera:

```
> ./runPruebaRender
```

Esto levantará un menú interactivo por consola que ofrecerá estrictamente las siguientes opciones numeradas:

1. **Desplazar cámara:** Permite al usuario ingresar las nuevas coordenadas enteras (X, Y) del centro de la cámara. **Inmediatamente al situarse**, el sistema ejecuta el filtro de visibilidad sobre los cuadrantes aledaños, selecciona el algoritmo de ordenamiento adecuado para el volumen de datos e imprime en pantalla el nombre del algoritmo utilizado y la cantidad de elementos procesados. Si la escena contiene elementos visibles ($N > 0$), la secuencia ordenada de IDs se guarda en el histórico. Si no hay elementos visibles, el fotograma se omite por completo del registro.
2. **Insertar elemento:** Permite añadir una nueva entidad en coordenadas específicas ingresadas por el usuario, validando las fronteras del tablero ($[0, 99]$), los rangos de z $[0, 0, 10, 9]$, y verificando el invariante de carga del cuadrante afectado (arrojando el mensaje de rechazo si se supera la densidad de 0.8). **Nota: Debe guardar el id mas alto asignado hasta el momento para garantizar la unicidad de los IDs.**

3. **Reiniciar el mapa:** Vacía por completo el estado de la memoria de todas las regiones y vuelve a ejecutar el paso de carga limpia del archivo de texto original de la cátedra, descartando inserciones o desplazamientos previos.
4. **Exportar histórico:** Retorna un archivo llamado `historico.txt` el cual contendrá el contenido completo del arreglo de arreglos almacenado en memoria. El archivo tendrá los IDs del sub-arreglo correspondiente separados por comas y separará obligatoriamente cada bloque de movimiento del siguiente mediante una línea vacía. Además de ignorar los -1, es decir, no aparecerán en el archivo de salida.
5. **Salir.**

4.1. Ejemplo Integrado de Ejecución e Historial

Para ilustrar el funcionamiento tenemos el siguiente ejemplo:

1. Archivo de Entrada Original (`objetos.txt`):

Suponga un archivo de configuración base con todas las entidades espaciales distribuidas en el tablero de 100×100 (mostrando aquí un fragmento con los primeros 5 objetos cargados):

```
101,5,4,2.5
102,8,6,1.0
103,6,4,3.0
104,15,5,2.5
105,7,3,0.5
.
.
.
```

Y se corre el programa `PruebaRender.kt` con un máximo de movimientos permitido de 3 (`runPruebaRender.sh 3`).

El historial de movimientos en memoria se inicializa como un arreglo de 3 filas y 720 columnas, lleno de '-1':

```
historial = [[-1, -1, -1, ..., -1], [-1, -1, -1, ..., -1], [-1, -1, -1, ..., -1]]
```

2. Simulación de Movimientos de la Cámara y Filtro de Visibilidad:

- **Movimiento 1:** Supongamos las coordenadas (3, 4). El filtro detecta 4 objetos visibles: 101, 102, 103 y 105. Tras aplicar el ordenamiento automático (`insertionSort`), el motor genera la secuencia ordenada de IDs: [105, 102, 101, 103] y se almacena en el histórico.
- **Movimiento 2:** Se desplaza la cámara a una esquina vacía del mapa. El filtro no detecta objetos visibles ($N = 0$). Este movimiento se omite por completo y no genera ninguna estructura en memoria.
- **Movimiento 3:** Supongamos las coordenadas (10, 4) El filtro detecta 5 elementos visibles: 101, 102, 103, 104 y 105. Aquí ocurre un **empate en el criterio principal** (Coordenada Z): los elementos 101 y 104 poseen exactamente $z = 2.5$. El motor aplica el **criterio de desempate por coordenada Y** de mayor a menor: como el elemento 104 está en $y = 5$ y el 101 está en $y = 4$, se ordena primero el objeto 104.

El resultado ordenado es: [105, 102, 104, 101, 103]. Esta secuencia se añade como la segunda fila del histórico.

3. Estado del Historial de Movimientos en Memoria:

Al finalizar los tres desplazamientos, el arreglo bidimensional nos quedaría:

```
historial = [[105, 102, 101, 103, -1, -1, ...], [105, 102, 104, 101, 103, -1, ...], [-1, -1, -1, -1, -1, -1, ...]]
```

4. Archivo de Salida Exportado (historico.txt):

Al seleccionar la opción 4 del menú, el sistema genera el archivo historico.txt de la siguiente manera:

```
105, 102, 101, 103  
  
105, 102, 104, 101, 103
```

Note que hay un espacio en blanco entre las dos secuencias ordenadas.

4.2. Requerimientos Técnicos y Entrega

- Todos los TADs deben modelarse como clases nativas de Kotlin solo haciendo uso de **arreglos** (Array).
- La compilación debe automatizarse mediante un archivo **Makefile**, el código debe estar documentado y se debe agregar un archivo README explicando el funcionamiento del sistema.
- La versión final debe estar contenida en un archivo comprimido con formato **.tar.xz** llamado **Proyecto_X_Y.tar.xz**, donde X e Y son los números de carné de los estudiantes.
- La entrega debe hacerse por la plataforma Classroom, antes de las 11:59 pm del día lunes 22 de junio de 2026.

Leonardo López / leonardolopez@usb.ve
Kevin Oporto / kevinoportoh@gmail.com
Junio 2026